C — Verified search and test-time compute (the Thinking tier)

0. Role in the system

This section informs the **Thinking tier**: the module that, for a task the worker (9B, VRAM) cannot solve in one pass, (1) samples N distinct *plans* (diversity at the plan level, not the token level), (2) implements each plan into a candidate patch, (3) scores every candidate with an **execution-based verifier cascade ordered by cost** (lint → type-check → tests → build → browser), (4) expands the survivors as a tree search guided by execution reward and the model's own value estimate, (5) **stops anytime** once one candidate passes every gate and a second, independently produced candidate agrees with it behaviourally, and (6) sizes the whole budget (N, children per expansion, iterations, depth) by an estimate of the task's difficulty.

After reading this pack the implementing agents must be able to build: the UCT/MCTS loop with LATS-style value and reflection; ToT-style plan proposal and state evaluation; CodeT/AlphaCode-style behavioural clustering of candidates on generated tests with the dual-agreement score as the "second agrees" criterion; self-consistency voting and its agreement-as-confidence signal for stopping; the coverage-vs-samples scaling law from Large Language Monkeys to size sample budgets and to predict the verifier gap; and the compute-optimal, difficulty-binned allocation rule from Snell et al. (sequential revision for easy tasks, parallel/tree search for hard ones).

1. Papers

1.1 Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models (Zhou et al., 2023; ICML 2024) — arXiv:2310.04406

Problem and claim. Prompting methods either act reflexively without considering alternatives (CoT, ReAct) or search over alternatives without using external feedback (ToT, RAP). LATS claims to be "the first general framework that synergizes the capabilities of LMs in reasoning, acting, and planning": Monte Carlo Tree Search over agent trajectories, with an LM-powered value function, environment feedback as reward, and verbal self-reflection stored in memory — no training.

Headline results. (v3/ICML text; v2 differs on one number, noted.) - HumanEval pass@1: GPT-3.5 — CoT 46.9, ReAct 56.9, Reflexion 68.1, ToT 54.4, RAP 63.1, **LATS 83.8**; GPT-4 — Base 80.1, Reflexion 91.0, **LATS 92.7** (v2 reported 94.4). - MBPP pass@1 (GPT-3.5): CoT 54.9, ReAct 67.0, Reflexion 70.0, ToT 65.8, RAP 71.4, **LATS 81.1**. - HotPotQA EM (GPT-3.5, $n = 5$, $k = 50$): ReAct 0.32, ReAct best-of-k 0.38, Reflexion 0.51, ToT(ReAct) 0.39, RAP(ReAct) 0.54, LATS(ReAct) 0.63, LATS($n = 3$) 0.58, LATS($n = 10$) 0.65, LATS(CoT+ReAct) 0.71; reasoning-only: CoT 0.34, CoT-SC 0.38, ToT 0.55, RAP 0.60, LATS(CoT) 0.62. - WebShop (GPT-3.5, 50 envs, $n = 5$, $k = 30$): score/SR — ReAct 53.8/28.0, ReAct best-of-k 59.1/32.0, Reflexion 64.2/35.0, **LATS 75.9/38.0**; IL 59.9/29.1, IL+RL 62.4/28.7, fine-tuning 67.5/45.0, human expert 82.1/59.6. - Game of 24 (GPT-3.5, $n = 5$, $k = 30$, depth 5): CoT 0.08, Reflexion 0.12, ToT 0.20, RAP 0.40, LATS(CoT) 0.44 (with $\lambda = 1$: 0.40). - Sample complexity (HotPotQA, Table 9): ReAct best $k = 250$: 0.42, $O(k)$; CoT-SC $k = 250$: 0.40; LATS $n = 1, k = 50$: 0.48; ToT $n = 5, k = 50$: 0.49, $O(kn)$, 210,215 tokens on success; RAP: 0.54, 176,500 tokens; LATS $n = 5, k = 50$: 0.63, 173,290 tokens. Avg nodes at $k = 50$: ToT 84.05, RAP 70.60, LATS 66.65.

Method — equations. Selection (Eq. 1, §3.2 / §4.2):

$$UCT(s) = V(s) + w\sqrt{\frac{\ln N(p)}{N(s)}}$$

s: child node; p: its parent; $N(\cdot)$: visit count; $V(s)$: value estimate of the subtree at s; w: exploration weight ($w = 1$ in all experiments; ablation 0.5 → 0.55 EM, 2.0 → 0.63 EM).

Value function (Eq. 2, §4.2, v3):

$$V(s) = \lambda \cdot \mathrm{LM}(s) + (1-\lambda) \cdot \mathrm{SC}(s)$$

$\mathrm{LM}(s)$: the LM's self-generated score for the trajectory at s (prompt ends "Thus the correctness score is s", integer 1–10); $\mathrm{SC}(s)$: self-consistency score, "how often an action is sampled at the same state" (no explicit formula in the text; cites Wang et al. 2022); $\lambda = 0.8$ for programming, 0.5 for Game of 24. (v2 of the paper used only $\mathrm{LM}(s)$.)

Backpropagation (§4.2), for the path $s_0, \ldots, s_l$ and terminal reward r:

$$N(s_i) \leftarrow N(s_i) + 1, \qquad V(s_i) \leftarrow \frac{V_{\mathrm{old}}(s_i)\,(N(s_i) - 1) + r}{N(s_i)}$$

Method — algorithm. Algorithm 1, LATS $\left(s_0, p_\theta, p_V, p_{\mathrm{ref}}, d, k, n, w\right)$: initial state, action generator, value function, reflection generator, depth limit, number of roll-outs, actions per expansion, exploration weight. 1. Initialise visit counts and values (v3 text: to one). 2. For each of k roll-outs, walk from the root for up to d steps: 1. **Expansion + evaluation**: if the current state is non-terminal, sample n actions from p_θ, obtain an observation from the environment for each, form n child states, score each with $\lambda p_V + (1-\lambda)\,\mathrm{SC}$, add them as children. 2. **Reflection**: if the state is terminal and unsuccessful, prompt p_{ref} with the trajectory and reward; store the (trajectory, reflection) pair; add it to the context of the actor and the value function in later iterations. 3.

Selection: move to $\arg\max_{s'} V(s') + w\sqrt{\ln N(s)/N(s')}$; stop the roll-out if the selected action is an output action. 3.

Backpropagation after each roll-out with the final reward. 4. **Simulation** (generic tasks): expand the selected node greedily on value until terminal; success ends the search. Programming instantiation (§5.2, App. D.2): each action is a *complete* candidate solution, so simulation is skipped; the LM first generates a synthetic suite of `assert` statements (4 tests in main text; App. D.2: 6 for GPT-3.5, 4 for GPT-4, following CodeT); observations are passed/failed tests plus compiler output; reward = percentage of generated tests passed; $n = 5$, $k = 8$, depth 8; the highest-value solution is run on the real tests for pass@1.

Hyperparameters and setup. $n = 5$, $w = 1$ everywhere; HotPotQA $k = 50$, $d = 7$ (App. D.1 says 6), 100 questions, 3 few-shot examples; WebShop $k = 30$, depth 15; programming $k = 8$, depth 8, $\lambda = 0.8$; Game of 24 $k = 30$, depth 5, $\lambda = 0.5$. Temperature: (not in fetched text — verify in paper App. D).

Ablations that matter. HotPotQA EM (Table 8/11): full LATS 0.63; no LM heuristic (environment reward only) 0.37 (−0.26); replace MCTS by DFS 0.42; no reflection 0.58; $d = 4$: 0.58; $w = 0.5$: 0.55. More children helps: $n = 3$ 0.58, $n = 5$ 0.63, $n = 10$ 0.65. Game of 24: $\lambda = 1$ (LM only) 0.40 vs $\lambda = 0.5$ 0.44.

Code / data / license. https://github.com/lapisrocks/LanguageAgentTreeSearch (MIT; dirs `hotpot/`, `programming/`, `webshop/`; flags `--n_generate_sample`, `--n_evaluate_sample`, `--iterations`).

Stated limitations. Higher compute, time and token cost than ReAct/Reflexion ("use for difficult tasks where performance matters more than efficiency"); assumes the environment can be reverted to earlier states (rollback); benchmarks simpler than real interactive environments; Table 9 vs Table 10 disagree (0.63 vs 0.61 at $k = 50$).

Use in our design. The Thinking tier's outer loop is LATS with the programming instantiation: node = candidate patch on a git worktree snapshot (rollback is free), reward = cascade pass fraction, $w = 1$, $n = 5$, $k \le 8$, depth ≤ 8. Replace the LM-only value with $\lambda \cdot \mathrm{LM} + (1-\lambda) \cdot \mathrm{SC}$ where SC is the behavioural-cluster size from CodeT (§2.1). Keep reflections in the Notebook (section A) so they persist across tasks, not just iterations. Because the worker is small, give the senior model the value-function prompt and the worker the expansion. PDF: https://arxiv.org/pdf/2310.04406 — Code: https://github.com/lapisrocks/LanguageAgentTreeSearch

1.2 Tree of Thoughts: Deliberate Problem Solving with Large Language Models (Yao et al., 2023; NeurIPS 2023) — arXiv:2305.10601

Problem and claim. LMs decode token by token, left to right, which fails on tasks needing exploration, lookahead or backtracking, where "initial decisions play a pivotal role". ToT generalises CoT: explore coherent *thoughts* as tree nodes, let the LM evaluate partial states, and run a classical search (BFS/DFS) over them.

Headline results. Game of 24 (GPT-4, 100 games): IO 7.3%, CoT 4.0%, CoT-SC($k = 100$) 9.0%, IO+Refine($k = 10$) 27%, IO best-of-100 33%, CoT best-of-100 49%, ToT($b = 1$) 45%, **ToT($b = 5$) 74%**; ~60% of CoT samples already fail after the first step. Creative Writing (GPT-4 coherence 1–10): IO 6.19, CoT 6.93, ToT 7.56; with iterative refine IO 7.67, ToT 7.91; humans preferred ToT over CoT 41 vs 21 of 100 pairs. Mini Crosswords (letter/word/game %): IO 38.7/14/0, CoT 40.6/15.6/1, ToT 78/60/20, ToT+best state (oracle) 82.4/67.5/35, ToT −prune 65.4/41.5/5, ToT −backtrack 54.6/20/5. Cost (App. B.3, Game of 24, generate/prompt tokens, $ per case): IO best-of-100 1.8k/1.0k $0.13; CoT best-of-100 6.7k/2.2k $0.47; ToT 5.5k/1.4k $0.74. Creative Writing: IO 0.9k/0.4k $0.06; CoT 0.9k/0.4k $0.07; ToT 4k/2.9k $0.32. "ToT could require 5–100 times more generated tokens than CoT."

Method — equations. (§3) p_θ: LM; x: input; z_i: i-th thought; state $s = [x, z_{1..i}]$. Thought generator, two strategies:

$$z^{(j)} \sim p_\theta^{\mathrm{CoT}}(z_{i+1} \mid s),\ j = 1..k \quad \text{(sample i.i.d.; rich thought spaces)}$$

$$\left[z^{(1)}, ..., z^{(k)}\right] \sim p_\theta^{\mathrm{propose}}\left(z_{i+1}^{(1..k)} \mid s\right) \quad \text{(propose in one context; constrained spaces, avoids duplicates)}$$

State evaluator $V(p_\theta, S)$, two strategies:

$$V(p_\theta, S)(s) \sim p_\theta^{\mathrm{value}}(v \mid s) \quad \text{(independent scalar or label sure/likely/impossible)}$$

$$V(p_\theta, S)(s) = \mathbb{1}\left[s = s^*\right],\ s^* \sim p_\theta^{\mathrm{vote}}(s^* \mid S) \quad \text{(vote across the frontier)}$$

Method — algorithm. Algorithm 1 ToT-BFS $(x, p_\theta, G, k, V, T, b)$:

```
S_0 <- {x}
for t = 1..T:
    S'_t <- { [s, z] : s in S_{t-1}, z in G(p_theta, s, k) }
    V_t  <- V(p_theta, S'_t)
    S_t  <- argmax_{S subset S'_t, |S| = b} sum_{s in S} V_t(s)
return G(p_theta, argmax_{s in S_T} V_T(s), 1)
```

Algorithm 2 ToT-DFS $(s, t, p_\theta, G, k, V, T, v_{th})$:

```
if t > T: record output G(p_theta, s, 1); return
for s' in G(p_theta, s, k):            # sorted by promise
    if V(p_theta, {s'})(s') > v_th:    # else prune
        DFS(s', t+1)
```

Task settings: Game of 24 — BFS, 3 thought steps (one equation each), $b = 5$, each thought valued 3 times with sure/maybe/impossible (code maps sure→20, likely→1, impossible→0.001 and sums, "ad hoc"); Creative Writing — depth 2, 5 plans + 5 votes, 5 passages + 5 votes, $b = 1$; Crosswords — DFS, ≤10 steps, prune when any remaining clue is "impossible", backtrack, cap 100 search steps, output deepest state.

Hyperparameters and setup. GPT-4 Chat Completions, temperature 0.7, May 5–16 2023. Baselines sampled 100× (Game of 24) / 10× (writing).

Ablations that matter. $b = 1$ → 45%, $b = 5$ → 74% (Game of 24); removing pruning 20→5% game success, removing backtracking 20→5% (Crosswords); best-of-100 CoT (49%) is "still much worse than exploring more nodes in ToT".

Code / data / license. https://github.com/princeton-nlp/tree-of-thought-llm (license not stated in fetched text).

Stated limitations. Unnecessary for tasks GPT-4 already solves; only three "relatively simple" tasks; needs more resources (GPT-4 cost) than sampling; GPT-4 auto-scoring for writing is noisy; DFS pruning heuristics are "critical" and imperfect; fine-tuning LMs for ToT-style decisions is future work.

Use in our design. Plan-level diversity = ToT's *propose* strategy: one prompt that emits k *distinct* plans in a single context (dedupe free), not k i.i.d. samples. Keep the frontier at b plans (start $b = 3$ locally, 5 when VRAM allows batching). Use the vote evaluator (senior model) to rank plans before implementation, and the value evaluator (3 samples, sure/likely/impossible) for cheap pre-execution pruning. Hard prune rule (DFS-style) when the cascade's first gate fails. PDF: https://arxiv.org/pdf/2305.10601 — Code: https://github.com/princeton-nlp/tree-of-thought-llm

1.3 Self-Consistency Improves Chain of Thought Reasoning in Language Models (Wang et al., 2022; ICLR 2023) — arXiv:2203.11171

Problem and claim. Greedy decoding of a single chain of thought is "repetitive and locally optimal". Since correct reasoning admits many paths that agree on the answer more than incorrect paths do, sample many paths and take the most consistent final answer. No training, no verifier, no extra annotation.

Headline results. CoT-greedy → self-consistency (40 paths), absolute accuracy: PaLM-540B — GSM8K 56.5→74.4, SVAMP 79.0→86.6, AQuA 35.8→48.3, StrategyQA 75.3→81.6, ARC-c 85.2→88.7, MultiArith 94.7→99.3, AddSub 91.9→93.7, ASDiv 74.0→81.9, CSQA 79.0→80.7. GPT-3 code-davinci-002 — GSM8K 60.1→78.0 (+17.9), SVAMP 75.8→86.8 (+11.0), AQuA 39.8→52.0 (+12.2), StrategyQA 73.4→79.8 (+6.4), ARC-c 83.6→87.5 (+3.9). LaMDA-137B GSM8K 17.1→27.7; UL2-20B GSM8K 4.1→7.3. Ensembles are far weaker: prompt-order ensemble 19.2, multi-prompt 18.6 vs 27.7 (LaMDA GSM8K); 3-model greedy ensemble 33.3–36.9 vs 74.4 (PaLM GSM8K). Beam search (UL2, AQuA): top beam 23.6/19.3/16.1 at 1/5/10; SC over beams 23.6/19.8/21.2; SC over samples 19.7/24.9/25.3 — diversity from sampling matters.

Method — equations. m sampled outputs $(\mathbf{r}_i, \mathbf{a}_i)$, reasoning path $\mathbf{r}_i$, answer $\mathbf{a}_i \in \mathbb{A}$. Marginalise over paths (majority vote):

$$a^* = \arg\max_a \sum_{i=1}^{m} \mathbb{1}(\mathbf{a}_i = a)$$

Optional weighting by the length-normalised path probability (Eq. 1):

$$P(\mathbf{r}_i, \mathbf{a}_i \mid \text{prompt}, \text{question}) = \exp\left(\frac{1}{K}\sum_{k=1}^{K} \log P(t_k \mid \text{prompt}, \text{question}, t_1, ..., t_{k-1})\right)$$

t_k: k-th token of $(\mathbf{r}_i, \mathbf{a}_i)$; K: its token count. Normalised weighted sum ≈ unweighted vote; unnormalised worse; weighted *average* (divide by count) much worse.

Method — algorithm. 1. Prompt with CoT exemplars. 2. Replace greedy decoding with sampling to obtain m diverse paths. 3. Parse each final answer; return the answer with the most votes (optionally weighted as above).

Hyperparameters and setup. 40 paths by default; UL2-20B and LaMDA-137B: $T = 0.5$, top-$k = 40$; PaLM-540B: $T = 0.7$, $k = 40$; GPT-3: $T = 0.7$, no top-k, max 128 tokens; outputs cut at the next "Q:". Accuracy "saturates quickly": 5–10 paths capture most of the gain.

Ablations that matter. Robust to imperfect prompts (LaMDA GSM8K greedy 17.1→14.9 with bad prompts, SC 23.4); zero-shot CoT PaLM 43.0→69.2; **calibration**: the fraction of sampled paths agreeing with the chosen answer "correlates highly with accuracy" on GSM8K (Fig. 5) — low agreement flags low confidence. Sample-and-rank improves accuracy but "much smaller" than SC.

Code / data / license. No code release; prompts in App. A.3. License n/a.

Stated limitations. Higher compute cost; models still produce incorrect or nonsensical rationales; requires a parseable final answer to vote over.

Use in our design. Two uses: (a) agreement fraction as the **stopping/confidence signal** — when the top behavioural cluster holds $\geq$ a set fraction of candidates, stop; when agreement is low, escalate budget or the senior model; (b) votes over *plans* before implementation (plan-level SC). Use $T \approx 0.6$–0.7 for the worker; 5–10 samples is the sweet spot locally, not 40. PDF: https://arxiv.org/pdf/2203.11171

1.4 CodeT: Code Generation with Generated Tests (Chen et al., 2022; ICLR 2023) — arXiv:2207.10397

Problem and claim. A code LM can sample many candidates but choosing the correct one is hard without human tests. CodeT has the same model generate tests, executes every candidate against every generated test, and ranks candidates by **dual execution agreement**: agreement with the tests *and* with other candidates.

Headline results. HumanEval pass@1/@2/@10 (baseline pass@1 / @10 / @100 in brackets): code-davinci-002 **65.8**/75.1/86.6 (47.0/74.9/92.1), code-davinci-001 50.2/58.9/75.8 (39.0/60.6/84.1), code-cushman-001 44.5/50.1/65.7 (33.5/54.3/77.4), InCoder-6B 20.6/27.6/37.1 (16.4/28.3/47.5), CodeGen-Mono-16B 36.7/44.7/59.3 (29.7/50.3/73.7). AlphaCode-C style clustering (no test agreement) on davinci-002: 55.1/64.1/84.4. MBPP davinci-002: baseline 58.1 → CodeT 67.7 pass@1. APPS (davinci-002, zero-shot)

pass@1 baseline→CodeT: Introductory 27.2→34.6, Interview 5.1→8.1, Competition 1.8→2.2; CodeContests 0.7→2.1 (pass@100 7.5→9.9). CodeT pass@2 ≈ baseline pass@10.

Method — equations. Candidates $X = \{x_1..x_N\}$, generated tests $Y = \{y_1..y_M\}$; x *passes* y if it runs without error and output matches. For a hypothetical inlier (x, y) (an x that passes y): S_y = set of tests x passes; S_x = set of solutions that pass *exactly* the same tests as x. Consensus set $S = \{(x, y) \mid x \in S_x, y \in S_y\}$, scored by (unnumbered, §3.2):

$$f(S) = |S_x| \cdot |S_y|$$

Implementation (App. A, Fig. 8) uses

$$f(S) = \sqrt{|S_x|} \cdot |S_y|$$

to damp the solution-count term ("sqrt" ≥ "linear" on HumanEval).

Method — algorithm. RANSAC-inspired: sample pairs (x, y) from $D = X \times Y$; if x passes y build the consensus set, score, repeat a fixed number of times. **Exhaustive version used in practice:** 1. Run every $x \in X$ on every $y \in Y$; record pass vector $\pi(x) \in \{0,1\}^M$ (0.1 s timeout per test). 2. Group solutions with identical $\pi(x)$; each group G is one consensus set with $S_x = G$, $S_y = \{y : \pi_y = 1\}$. 3. Score $f(S) = \sqrt{|S_x|}\,|S_y|$; rank sets. 4. For pass@k, take one solution from each of the top-k sets; if fewer than k sets, cycle from the top. Test generation: context = function stub + docstring with example I/O removed; instruction = `pass` body + comment "check the correctness of " + an opening `assert`; keep the first 5 syntactically valid asserts per sample.

Hyperparameters and setup. Solutions per problem: HumanEval/MBPP 100, APPS 50, CodeContests 1,000; test samples: 100 (HumanEval/MBPP), 50 (APPS/CodeContests); $T = 0.8$, top-$p = 0.95$, max length 300; stop sequences `\nclass`, `\ndef`, `\n#`, `\nif`, `\nprint`; generated tests per problem ≈380–410 (Codex, InCoder), 55.6 (CodeGen); statement/branch coverage of generated tests 95.7/98.5 (davinci-002), 78.2/78.6 (CodeGen).

Ablations that matter. Scoring variants, HumanEval pass@1 (Table 13; baseline / $|S_x|$ only / $|S_y|$ only / CodeT): cushman 33.5/41.2/29.9/44.5; davinci-001 39.0/44.4/35.0/50.2; davinci-002 47.0/55.9/58.4/65.8 — tests-only is *below baseline* for weak models; both factors needed. Few tests still help (Table 5, davinci-002): 1 assert per sample and 10 samples → 56.5; 5 asserts × 100 samples → 65.8; gains flatten at ≥3 asserts and ≥50 samples. De-duplication: slight, inconsistent; removing trivial solutions: little gain.

Code / data / license. https://github.com/microsoft/CodeT (license not stated in paper).

Stated limitations. Only for executable code; extra cost for test generation; failure cases: low-quality/"toxic" tests (passed by a wrong candidate, failed by the reference), partially correct solutions, model misreading the spec; 53/164 HumanEval problems (cushman) had a correct solution outside the top set, ~20% of those due to ambiguous descriptions, uncovered corner cases or missing imports.

Use in our design. The cascade's *tests* gate runs both the repo's tests and worker-generated asserts; candidates are clustered by their pass vector; cluster score $\sqrt{|S_x|}\,|S_y|$ ranks them, and "**a second candidate agrees**" is operationalised as $|S_x| \geq 2$ in the top cluster. Generate 3–5 asserts per sample, 5–10 samples (not 100), since the 9B worker's test accuracy will be lower than Codex's — track toxicity (§2.5). PDF: https://arxiv.org/pdf/2207.10397 — Code: https://github.com/microsoft/CodeT

1.5 Competition-Level Code Generation with AlphaCode (Li et al., 2022; Science 378:1092–1097) — arXiv:2203.07814

Problem and claim. Competitive programming needs algorithmic reasoning, not just instruction-to-code translation. AlphaCode = curated CodeContests dataset + large encoder–decoder transformers + **large-scale sampling, then filtering on example tests and clustering by behaviour** to pick ≤10 submissions.

Headline results. Codeforces: 10 simulated contests (>5,000 participants each), average top 54.3% ranking (best 48.4%, worst 77.2%), estimated rating 1238 ("top 28%" of recent users). CodeContests 10@k (validation / test): 41B — 10@1K 16.9/15.6, 10@10K 23.9/23.2, 10@100K 28.2/27.7, 10@1M 31.8/–; **41B + clustering** 21.0/16.4, 26.2/25.4, 31.8/29.6, 34.2/–; 9B 16.9/14.3, 22.6/21.5, 27.1/25.8, 30.1/–. 1B ablation 10@1K/10K/100K/1M: +GOLD 12.4/17.3/21.5/24.2 → +Clustering 12.2/18.0/24.1/28.4. Solve rate "scales log-linearly with more samples"; larger models have steeper slopes. APPS (1B, 1000 samples, no filtering): Intro 17.67, Interview 5.24, Competition 7.06; 50k samples/5 attempts: 20.36/9.66/7.75.

Method — equations. Metric $n@k$: percentage of problems solved when n submissions are chosen from k samples; solved iff any of the n passes all hidden tests. pass@k = $n@k$ with unlimited n, the upper bound for a perfect selector. (No further equations in the sampling/filtering sections.)

Method — algorithm. (§4.4–4.6, §5.3.5) 1. **Sample** up to 10^6 programs per problem, half Python, half C++, temperature 0.25 (with tempering + GOLD) or 0.12 (tempering only); nucleus/top-k gave no gain; metadata conditioning: random tags from the 50 most popular, ratings uniform in 800–3500. 2. **Filter** on the example tests in the problem statement: removes ≈99% of samples (<1% pass; Table 9); ≈10% of problems have no passing sample (1B: 87.18% of problems have ≥1); "filtering 99% of a million samples still leaves thousands of samples per problem". 3. **Cluster**: a separate *test-input generation model* (same architecture, initialised from the GitHub-pretrained checkpoint, trained to predict test inputs from problem descriptions using example, hidden and generated test inputs) produces new inputs for the unseen problem (not guaranteed valid; imperfect inputs still separate programs). Run every surviving program on these inputs; programs with identical outputs form one cluster ("behaviour" = output pattern on generated inputs). 4. **Select**: sort clusters by size; submit one program from each cluster, largest first; with fewer than 10 clusters, cycle back to the largest, skipping already-submitted samples; ≤10 submissions. Dataset-side generated tests (§3.2.1): mutate existing inputs (bit flips, integer ±, character swaps), run 30 correct solutions, keep inputs where all agree; ≤200 tests or 10 CPU-hours per problem; false-positive rate 62% (raw) → 4%; "FP or slow" 88% → 46%; APPS FP 60%, HumanEval FP 30%.

Hyperparameters and setup. Models 300M–41B; multi-query attention: 4.74 vs 0.37 samples/TPU-second (same 10@10K 17.3 vs 17.0); tempering $T = 0.2$ at training; GOLD offline-RL objective; value conditioning/prediction. Cumulative 1B ablation at 10@100K: +MLM 17.0 → +Tempering 18.7 → ... +Value 20.2 → +GOLD 21.5 → +Clustering 24.1.

Ablations that matter. Fig. 8: random pick without filtering stays flat as samples grow; filtering + clustering "clearly enable scaling" but leave "a large gap" to pass@k; clustering 10@1M 31.8→34.2 (41B val), 10@100K 27.7→29.6 (test). Clustering can slightly hurt at tiny budgets (1B 10@1K 12.4→12.2).

Code / data / license. Dataset: https://github.com/deepmind/code_contests (license not in fetched text). Model weights not released.

Stated limitations. Hundreds of petaFLOPS-days; slow-but-correct "slow positives" remain (46%); generated inputs may be invalid; gap to pass@k shows selection is the bottleneck; false positives in public datasets.

Use in our design. Three transfers: (i) **cheapest filter first** — the example-test / lint gate removes most candidates before anything expensive; (ii) **behavioural clustering with model-generated inputs** when the repo has no tests (generate inputs, not asserts, and cluster on outputs — a weaker but assert-free agreement signal); (iii) **largest-cluster-first submission order** when several candidates survive. Scale: tens of samples, not 10^6; the log-linear law says each doubling buys a fixed increment. PDF: https://arxiv.org/pdf/2203.07814 — Data: https://github.com/deepmind/code_contests

1.6 Large Language Monkeys: Scaling Inference Compute with Repeated Sampling (Brown et al., 2024; arXiv/CoRR) — arXiv:2407.21787

Problem and claim. Inference compute is usually spent on one attempt. Draw many independent samples at positive temperature and pick with a verifier: **coverage** (fraction of problems with ≥1 correct sample) rises with samples over four orders of magnitude, often following an exponentiated power law; gains become real only where a verifier can find the correct sample (code, formal proofs); otherwise majority vote / reward models plateau after a few hundred samples.

Headline results. SWE-bench Lite, DeepSeek-Coder-V2-Instruct + Moatless Tools: 15.9% (1 sample) → **56%** (250 samples) vs 43% prior single-attempt SOTA. CodeContests, Gemma-2B: pass@1 0.02% → pass@10k 7.1% (>300×). MATH, Pythia-160M: 0.27% → 57%. MATH, Llama-3-8B-Instruct: coverage 82.9% at 100 → 98.44% at 10,000 samples, but best verifier only 40.50% → 41.41%. Pythia on CodeContests: zero coverage at any budget. Cost (SWE-bench Lite): 5 DeepSeek attempts 29.62% solved at 1× ($0.0072/attempt, $10.8 total) vs 1 GPT-4o attempt 24.00% at 3.6× ($0.13) vs 1 Claude 3.5 Sonnet attempt 26.70% at 4.7× ($0.17). At fixed FLOPs, Llama-3-8B-Instruct beats 70B on MiniF2F, GSM8K, MATH; 70B wins on CodeContests.

Method — equations. Coverage / unbiased pass@k (Chen et al.): N samples per problem, C_i correct for problem i,

$$\text{pass@}k = \frac{1}{\#\text{problems}} \sum_{i=1}^{\#\text{problems}} \left(1 - \frac{\binom{N-C_i}{k}}{\binom{N}{k}}\right)$$

Scaling law (§3): coverage c vs sample count k,

$$\log c \approx a\, k^b \quad \Leftrightarrow \quad c \approx \exp\left(a\, k^b\right), \qquad a, b \in \mathbb{R} \text{ fitted}$$

Fit quality: mean ± sd of error between curve and fit over 100 log-spaced points (Fig. 5); "not as exact as training scaling laws (most strikingly on MiniF2F-MATH)". Fitted a, b values: (not in fetched text — verify in paper §3/App.). Family invariance (Fig. 6): shifting curves within a model family to pass through $(1, c)$ overlays them, so the sample multiplier needed to go from c to c' is roughly constant within a family. Inference cost (§2.3, dense Llama-3):

$$\text{FLOPsPerToken}(L) \approx 2\,(\text{NumParameters} + 2 \cdot \text{NumLayers} \cdot \text{TokenDim} \cdot L)$$

$$\text{TotalInferenceFLOPs} \approx \sum_{t=1}^{P} \text{FLOPsPerToken}(t) + \sum_{t=1}^{D} \text{FLOPsPerToken}(t+P) \cdot \text{NumCompletions}$$

P: prompt tokens; D: decode tokens per completion; L: context length.

Method — algorithm. 1. For each problem sample N completions (up to 10,000; SWE-bench Lite ≤250 full agent trajectories). 2. Verify: code → run tests (pass@k = coverage); Lean → proof checker; math → majority vote, reward-model best-of-N (ArmoRM-Llama3-8B-v0.1), or reward-weighted majority. 3. Report coverage and, separately, the selector's success rate; the difference is the **verifier gap**.

Hyperparameters and setup. Models: Llama-3-8B(-Instruct), Llama-3-70B-Instruct, Gemma-2B/7B, Pythia-70M...12B, DeepSeek-Coder-V2-Instruct. Problems: GSM8K 128, MATH 128, MiniF2F-MATH 130 (Lean 4, $T = 0.5$, no nucleus, 5 few-shot), CodeContests 122 Python3 test problems, SWE-bench Lite. Temperatures/top-p for GSM8K/MATH/CodeContests: (not in fetched text — verify in paper App. A).

Ablations that matter. Verifier methods on GSM8K/MATH "plateau around 100 samples" while coverage "eventually exceeds 95%"; 11.3% of SWE-bench Lite problems have flaky test suites; 35/122 CodeContests problems have reference solutions failing their own tests (false negatives); one GSM8K label is wrong.

Code / data / license. https://github.com/ScalingIntelligence/large_language_monkeys; data https://huggingface.co/datasets/ScalingIntelligence/monkey_business (license not in fetched text).

Stated limitations. Only temperature for diversity; single-turn, independent attempts (no learning from earlier samples — proposed future work, with verifier feedback); FLOPs ignore system efficiency; imperfect/flaky verifiers; unstructured-task verification open.

Use in our design. (a) Size budgets with the law: fit a, b per task class from the project's own history and spend samples only while the predicted coverage gain per unit cost beats escalation to the senior; (b) always log coverage *and* selected-success so

the verifier gap is visible; (c) the 8B-vs-70B result justifies many worker samples before one senior call on verifiable tasks; (d) run repeated-sampling in an agent loop (their SWE-bench setup) — attempts are whole trajectories, each ending in the cascade. PDF: https://arxiv.org/pdf/2407.21787 — Code: https://github.com/ScalingIntelligence/large_language_monkeys

1.7 Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters (Snell et al., 2024; ICLR 2025 oral) — arXiv:2408.03314

Problem and claim. Given a fixed extra inference budget on a hard prompt, how much can a model improve, and how does that trade against pretraining? Two mechanisms: (1) search over outputs against a process reward model (PRM) — best-of-N weighted, beam search, lookahead; (2) sequentially revising the proposal distribution. Their relative value "varies significantly depending on the difficulty of the prompt", so allocate compute per prompt by difficulty ("compute-optimal").

Headline results. MATH, PaLM 2-S*: compute-optimal allocation improves test-time efficiency by "more than 4×" over best-of-N (Abstract; §8 says 2–4×): search — matches best-of-N at 64 generations with 16; revisions — matches 256-sample best-of-N with 64. FLOPs-matched: on easy/intermediate questions test-time compute beats a ~14× larger pretrained model; on the hardest questions or at high inference load, pretraining wins.

Method — equations. Compute-optimal allocation (Eq. 1, §3.1):

$$\theta^*_{q,y^*(q)}(N) = \arg\max_{\theta} \left(\mathbb{E}_{y \sim \mathrm{Target}(\theta,N,q)}\left[\mathbb{1}_{y=y^*(q)}\right]\right)$$

θ: test-time strategy hyperparameters (e.g. search type, beam width, sequential/parallel ratio); N: budget; q: prompt; $\mathrm{Target}(\theta,N,q)$: output distribution induced by the strategy; $y^*(q)$: correct answer. Since y^* is unknown at deployment, approximate θ^* as a function of a **difficulty bin** of q: oracle difficulty = base-model pass@1 over 2048 samples, binned into 5 quantiles; model-predicted difficulty = mean PRM final-answer score over the same samples, binned (App. C; "requires applying a non-trivial amount of test-time compute itself"). Pretraining-vs-inference FLOPs (§7): $X = 6ND_{\text{pretrain}}$, $Y = 2ND_{\text{inference}}$, $R = D_{\text{inference}}/D_{\text{pretrain}}$, tested $R \in \{0.16, 0.79, 22\}$.

Method — algorithm. *Best-of-N weighted*: sample N answers; score each with the PRM (last-step score used as the answer score — beats min and product); sum scores per distinct final answer; pick the max. *Beam search* (N beams, width M): 1. Sample N first-step candidates. 2. Score each step by PRM reward-to-go. 3. Keep the top N/M. 4. From each, sample M next-step proposals (back to N); repeat 2–4 until a solution ends or 40 rounds; finish with best-of-N weighted over the N results. *Lookahead search*: at each step roll out k further steps at temperature 0, score by the PRM at the rollout end; beam = $k=0$; cost $N(k+1)$; generally underperforms at equal budget. *Sequential revisions*: revision model (PaLM 2-S* finetuned; data: 64 parallel samples per MATH train question at higher temperature, trajectories ≤5 steps stopping at first correct, each correct answer prefixed by 0–4 incorrect ones chosen by character edit distance, uniformly sampled count); at test time generate a chain of revisions; select by verifier or **sequential majority vote** — naive "take last" turns ~38% of correct answers back into incorrect ones. Combine with parallel chains via a sequential-to-parallel ratio. *PRM*: trained without human labels on PaLM 2-S* samples; per-step soft labels from 16 Monte Carlo rollouts; BCE loss; AdamW lr 3e-5, batch 128, dropout 0.05, betas (0.9, 0.95); early stopping on a 10% split; beats an ORM, gap grows with N. PRM800K (GPT-4 samples) did not transfer.

Hyperparameters and setup. MATH test; PaLM 2-S*; 4-shot step-by-step prompt; beam width $M=4$ in the difficulty figure; max 40 expansion steps; budgets 4–256 generations; difficulty from 2048 samples; 5 bins.

Ablations that matter. Beam search beats best-of-N at small budgets and on medium/hard bins (levels 3–4); on easy bins (1–2) beam search *degrades* with budget ("over-optimizing the PRM") while best-of-N does not; level 5: neither progresses. Revisions: sequential beats parallel under both verifier and majority selection; easy questions best with purely sequential compute, hard ones with a mix; the ideal ratio shifts toward parallel as difficulty rises and toward sequential at low budgets. Online (ReST-EM) training of the revision model made more revisions *hurt* (spurious correlations). Exact accuracies per budget: (not in fetched text — verify in paper Fig. 3, 6, 8).

Code / data / license. No code release found in fetched text.

Stated limitations. Difficulty estimation is itself costly; search and revisions not combined (future work); critique-and-revise not studied; small gains on the hardest problems; only MATH and PaLM 2-S*.

Use in our design. The **budget schedule**: bin tasks by a cheap difficulty proxy (worker pass-rate on the cheap gates over a small probe batch, or the senior's predicted score), then: easy → short sequential revision chain with few parallel plans; medium/hard → wider parallel plan sampling + tree expansion; hardest → escalate to senior or ask the user, since extra worker compute buys little. Use cascade pass-fraction as the "PRM"; keep "last-gate score" aggregation. Revision prompts include ≤4 prior failed candidates with their failing gate output (their data recipe). PDF: https://arxiv.org/pdf/2408.03314

2. Design extraction for the module

2.1 Algorithms to implement (consolidated pseudocode)

```
THINK(task, repo, cfg) -> Candidate | Escalate
  # --- 0. difficulty-binned budget (Snell §3.2, §5.3, §6.2) ---
  probe   <- sampleCandidates(worker, task, n=cfg.probeN)          # cheap: plan+patch, gates up to 'tests'
  d       <- bin(passFraction(probe), cfg.bins)                    # 5 bins; bin 5 = no probe passes lint/typecheck
  B       <- cfg.schedule[d]  # {N_plans, b_keep, n_children, k_iters, depth, seq_ratio, maxCost}
  if d == 5 and cfg.escalateHardest: return Escalate(reason="probe coverage 0")

  # --- 1. plan-level diversity (ToT propose strategy) ---
  plans   <- proposePlans(senior_or_worker, task, k=B.N_plans)     # ONE prompt, k distinct plans
  plans   <- dedupe(plans, bySignature)                            # drop near-identical plans
  scores  <- votePlans(senior, task, plans, votes=B.planVotes)     # ToT vote evaluator / plan-level self-consistency
  frontier<- topB(plans, scores, B.b_keep)

  # --- 2. LATS over implementation candidates ---
  root <- Node(state=repo.snapshot(), plan=null, N=1, V=0)
  for p in frontier: root.addChild(Node(plan=p, state=root.state))
  memory.reflections <- notebook.recall(task)                      # section A
  for iter in 1..B.k_iters:
     leaf <- root
     while leaf.children: leaf <- argmax_c  c.V + B.w * sqrt(ln(leaf.N) / c.N)      # UCT, w = 1
     if leaf.depth >= B.depth: backprop(leaf, leaf.reward); continue
     kids <- expand(worker, leaf, n=B.n_children, ctx=memory.reflections + leaf.failures)  # n patches
     for c in kids:
        c.state  <- apply(leaf.state, c.patch)                      # git worktree, rollback free
        c.gates  <- CASCADE(c.state, cfg.gates)                     # early exit on first failing gate
        c.reward <- rewardFromGates(c.gates)                        # §2.2
        c.sig    <- passVector(c.gates.tests)                       # bitmask over repo + generated tests
        c.V      <- B.lambda * lmScore(senior, c) + (1 - B.lambda) * scScore(c, kids ∪ pool)   # LATS Eq.2
        pool.add(c)
        if c.reward < 1 and c.isTerminal: c.reflection <- reflect(worker, c); memory.reflections.add(c.reflection)
     backprop(path(root..kids), max(c.reward))                      # LATS backprop
     # --- 3. anytime stopping (CodeT / AlphaCode / self-consistency) ---
     clusters <- clusterByPassVector(pool)                          # exhaustive CodeT
     top      <- argmax_S sqrt(|S.x|) * |S.y|
     if allGatesPassed(top) and |top.x| >= 2:  return pick(top)      # one passes everything AND a second agrees
     if allGatesPassed(top) and agreement(top, pool) >= cfg.agreeFrac: return pick(top)
     if cost(pool) > B.maxCost: break
  # --- 4. fallback ordering ---
  if any(allGatesPassed): return largestClusterFirst(clusters)       # AlphaCode submission order
  return Escalate(best=top, reflections=memory.reflections)

CASCADE(state, gates):               # cheapest first; stop at first failure
  results <- []
  for g in [lint, typecheck, unitTests(repo + generated), build, browser]:
     r <- run(g, state, timeout=g.timeout)
     results.append(r)
     if not r.passed: break
  return results

GENERATE_TESTS(worker, task, fnStubs):    # CodeT prompt shape
  for s in 1..cfg.testSamples:              # 5-10 locally
     out <- worker(stub + docstring(no examples) + "pass\n# check the correctness of <fn>\nassert ")
     keep first cfg.assertsPerSample (3-5) syntactically valid asserts
  (no repo tests?) -> also GENERATE_INPUTS and cluster on outputs (AlphaCode)

REVISE(worker, candidate):              # Snell revision data recipe, used for easy bins (seq_ratio high)
  ctx <- up to 4 prior failed candidates (closest by edit distance) + their failing gate output
  return worker(task, ctx, "produce a corrected patch")
```

2.2 Equations and quantities the implementation must compute

1. **UCT** (LATS Eq. 1): $UCT(s) = V(s) + w\sqrt{\ln N(p)/N(s)}$, $w = 1$.
2. **Node value** (LATS Eq. 2): $V(s) = \lambda \mathrm{LM}(s) + (1-\lambda)\,\mathrm{SC}(s)$, $\lambda = 0.8$ (LATS programming); $\mathrm{LM}(s) \in [0,1]$ = senior's 1–10 score / 10; $\mathrm{SC}(s) = |S_x(s)|/|\text{pool}|$ (size of s's behavioural cluster, normalised).
3. **Backprop**: $N(s) \mathrel{+}= 1$; $V(s) \leftarrow \big(V_{\text{old}}(s)\,(N(s)-1) + r\big)/N(s)$.
4. **Cascade reward** (LATS programming reward generalised): with gates $g_1..g_G$ in cost order and per-gate pass fraction $f_j \in [0,1]$ (f_j = fraction of tests passed for the tests gate, 0/1 otherwise),

$$r = \frac{1}{G}\sum_{j=1}^{G} f_j \cdot \prod_{i<j} \mathbb{1}[f_i = 1]$$

   i.e. credit accrues only up to the first failed gate; $r = 1$ iff all gates pass.
5. **Dual execution agreement** (CodeT): $f(S) = \sqrt{|S_x|}\cdot|S_y|$ per pass-vector cluster.
6. **Agreement / confidence** (Self-Consistency calibration): $\text{agree} = \max_S |S_x|/|\text{pool}|$; stop when $\text{agree} \geq$ threshold and the top cluster passes all gates.

7. **Coverage estimate** (Monkeys): $\text{pass@}k = 1 - \binom{N-C}{k} / \binom{N}{k}$ per task; aggregate per task class; fit $\log c \approx ak^b$ on the project's history to predict the marginal gain of the next doubling of samples.
8. **Difficulty bin** (Snell): quantile of probe pass-rate (oracle-style) or of senior-predicted score (model-predicted-style); 5 bins.
9. **Compute accounting** (Monkeys §2.3): $\text{FLOPsPerToken}(L) \approx 2\left(\text{params} + 2 \cdot \text{layers} \cdot d_{\text{model}} \cdot L\right)$; locally use wall-clock seconds and tokens as the cost unit, logged per node.
10. **Budget comparison** (Snell §7): test-time compute preferred when $R = D_{\text{inference}}/D_{\text{pretrain}} \ll 1$ and the task is not in the hardest bin — for a single user $R \ll 1$ always holds, so the only reason to skip search is the hardest bin.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```ts
type GateName = "lint" | "typecheck" | "tests" | "build" | "browser";

interface GateResult {
  gate: GateName; passed: boolean; fraction: number;      // fraction in [0,1]; tests gate = passed/total
  durationMs: number; stdoutTail: string; stderrTail: string;
  testVector?: Uint8Array;   // 1 bit per test (repo tests ++ generated asserts), CodeT pass signature
}

interface Plan { id: string; text: string; signature: string; votes: number; }   // ToT propose + vote

interface SearchNode {
  id: string; parentId: string | null; depth: number;
  plan: Plan; patch: string;                    // unified diff against parent snapshot
  snapshotRef: string;                          // git worktree / stash id (rollback)
  N: number; V: number;                         // LATS visit count and value
  reward: number;                               // cascade reward r (eq. 4)
  lmScore?: number; scScore?: number;           // components of V (lambda mix)
  gates: GateResult[]; terminal: boolean;
  reflection?: string;                          // LATS verbal reflection on failure
  tokensIn: number; tokensOut: number; wallMs: number;
  children: string[];
}

interface Cluster { passVector: string /*hex*/; memberIds: string[]; testsPassed: number;
                    score: number /* sqrt(|Sx|)*|Sy| */ ; allGates: boolean; }

interface BudgetSchedule {            // indexed by difficulty bin 1..5 (Snell)
  N_plans: number; b_keep: number; n_children: number; k_iters: number; depth: number;
  seq_ratio: number;                  // fraction of children produced as revisions vs fresh samples
  planVotes: number; lambda: number; w: number; maxCostMs: number; maxTokens: number;
}

interface ThinkRun {                  // one JSONL record per THINK call: feeds section D (self-training) and H (routing)
  taskId: string; difficultyBin: number; schedule: BudgetSchedule;
  nodes: SearchNode[]; clusters: Cluster[];
  outcome: "solved" | "escalated" | "budget_exhausted";
  coverage: number;                   // any node with r == 1 ?
  selectedId?: string; selectedCorrect?: boolean;   // filled later by user acceptance → verifier gap
  generatedTests: { source: "worker" | "senior"; asserts: string[]; toxic?: boolean }[];
}

interface CoverageFit { taskClass: string; a: number; b: number; samplesSeen: number; rmse: number; }  // log c = a k^b
```

File layout: `think/runs/<taskId>.jsonl` (ThinkRun), `think/tests/<taskId>/*.test.(ts|py)` (generated asserts, kept for regression), `think/coverage.json` (CoverageFit[]), reflections appended to the Notebook (section A).

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Quantity	Default	Source
Exploration weight w	1.0 (0.5 hurt, 2.0 neutral)	LATS Tables 8/11
Children per expansion n	5 (3 → −0.05 EM, 10 → +0.02)	LATS Table 3
Iterations k (programming)	8; depth 8	LATS App. D.2
λ in $V(s)$	0.8 (programming), 0.5 (Game of 24)	LATS v3
LM value prompt	integer 1–10, "Thus the correctness score is s"	LATS App.
Generated tests per problem	4 (GPT-4) / 6 (GPT-3.5)	LATS main/App. D.2
Plan breadth b	5 (74%) vs 1 (45%) on Game of 24; 3 locally	ToT Fig. 3
Value samples per thought	3; labels sure/likely/impossible (code: 20/1/0.001)	ToT §4.1, repo
Plan votes	5	ToT §4.2
Sampling temperature	ToT 0.7; SC 0.5–0.7, top-k 40; CodeT 0.8/top-p 0.95; AlphaCode 0.25; Monkeys 0.5 (Lean)	respective papers

Quantity	Default	Source
Self-consistency paths	40 in paper; 5–10 capture most gain	SC §3.5
Asserts kept per test sample	5; gains flatten at ≥3 asserts × ≥50 samples	CodeT App. A, Table 5
Test execution timeout	0.1 s per test (CodeT); raise to seconds for real suites	CodeT App. A
Cluster score	$\sqrt{\lvert S_x \rvert} \cdot \lvert S_y \rvert$	CodeT App. A
Submission order	largest cluster first, cycle if fewer than n	AlphaCode §4.6
Example-test filter	removes ≈99% of samples	AlphaCode §4.5
Verifier plateau	≈100 samples for majority/RM on math	Monkeys §4.1
Agent attempts cap	250 trajectories (SWE-bench Lite) → start at 8–16 locally	Monkeys
Beam width M / max steps	4 / 40	Snell §5
Prior failures in revision context	0–4, nearest by edit distance	Snell App. H
Difficulty bins	5 quantiles; 2048 samples in paper → 4–8 probes locally	Snell §3.2
Strategy per bin	easy bins: sequential revisions, few parallel; medium/hard bins: parallel plans + tree; hardest: escalate	Snell §5.3, §6.2

Suggested local schedule (derived, not from a paper): bin 1: $N_{\text{plans}} = 1$, revisions ≤3, no tree; bin 2: 2 plans, $n = 3$, $k = 2$; bin 3: 3 plans, $n = 5$, $k = 4$, depth 3; bin 4: 5 plans, $n = 5$, $k = 8$, depth 5, senior scores values; bin 5: escalate to senior for the plan, worker implements, or ask the user.

2.5 Evaluation: how to know the module works (metrics, from the papers)

- **Coverage vs. selected success** per task class (Monkeys): log both; the gap is the verifier gap. Target: selected ≥ 0.9 × coverage on tasks with repo tests.
- **pass@k curve and its fit** $\log c \approx ak^b$ (Monkeys Fig. 5): refit monthly; a flattening curve means sampling is no longer buying coverage → spend on senior/escalation instead.
- **n@k** (AlphaCode): fraction of tasks solved with ≤n user-visible candidates from k internal samples; keep $n = 1$ (one proposal to the user) and track $1@k$ vs k.
- **Success vs. nodes visited** (ToT Fig. 3a, LATS Table 10): plot solve rate against total nodes; LATS should beat best-of-k at equal node count (0.61 vs 0.42 at HotPotQA $k = 50$ equivalents).
- **Generated-test quality** (CodeT §4.3): accuracy = fraction of generated asserts the accepted solution passes; toxicity = fraction passed by a rejected candidate and failed by the accepted one; statement/branch coverage of generated asserts (CodeT: ~95%/98% for Codex, 78% for CodeGen).
- **False-positive and slow-positive rate** of the cascade (AlphaCode Table 2): sample accepted candidates and have the senior/user audit them; CodeContests went 62% → 4% FP by adding generated tests.
- **Agreement calibration** (SC Fig. 5): bin runs by agreement fraction; accuracy must rise monotonically with agreement or the stopping rule is miscalibrated.
- **Cost per solved task**: tokens, wall-clock and node counts (ToT App. B.3; LATS Table 9); the Thinking tier must be reserved for tasks where the fast path failed (ToT: “unnecessary for many existing tasks”).
- **Difficulty-bin ablation** (Snell Fig. 3/7): per bin, compare sequential-revision vs parallel-plans vs tree; the schedule is right when each bin’s chosen strategy is the per-bin winner.

2.6 Known pitfalls and failure modes (from the papers’ limitations)

- **Verifier over-optimisation**: beam search degrades on easy problems as budget grows (Snell); the LM-only value without environment reward drops HotPotQA EM 0.63 → 0.37 (LATS) — never let the LM score override a failed gate.
- **Toxic/low-quality generated tests** (CodeT): a wrong candidate can pass a wrong assert; tests-only scoring is below baseline for weak models — always combine with cross-candidate agreement, and track toxicity.
- **False positives from weak suites**: APPS 60%, HumanEval 30% FP without extra tests (AlphaCode); 11.3% flaky suites in SWE-bench Lite (Monkeys) — rerun flaky gates, quarantine flaky tests, treat “passes” on tiny suites as weak evidence.
- **Slow positives** (AlphaCode 46%): add timeouts/perf budgets to the tests gate.
- **Revision regressions**: naive “take the last revision” flips ~38% of correct answers to incorrect (Snell) — select among the chain by the cascade, never by recency.
- **Diversity collapse**: beam/greedy variants give worse self-consistency than sampling (SC Table 6); duplicate plans waste budget (ToT propose strategy exists for this).
- **Rollback assumption** (LATS): every node must restore an exact repo state — use git worktrees, never in-place edits.
- **Cost blow-up**: ToT 5–100× CoT tokens; LATS 173k tokens per HotPotQA success — cap by `maxCostMs`/`maxTokens` and by the coverage fit’s predicted marginal gain.
- **Hardest bin is a sink** (Snell level 5, Pythia on CodeContests): no amount of worker sampling helps when pass@1 ≈ 0; escalate instead.
- **Difficulty estimation cost** (Snell): 2048 samples is impossible locally; the probe batch itself must be reused as the first search samples.

- **Internal inconsistencies** in sources: LATS 0.63 vs 0.61 at $k = 50$, depth 7 vs 6, 94.4 vs 92.7 across versions; AlphaCode text vs Table 8 — do not treat any single number as precise.

2.7 Open questions this section does not settle

- The right λ when the reward is execution-dominated: LATS's 0.8 weights the LM score heavily because its tests were few; with a full cascade the LM score may deserve far less weight (or only tie-breaking).
- How to define "a second agrees" for **patches to a repo** rather than functions: pass-vector equality on repo + generated tests is proposed; AST-normalised diff equality or output clustering on generated inputs (AlphaCode) are alternatives whose precision locally is unknown.
- Whether the 9B worker generates asserts with high enough accuracy (CodeT shows weak models' tests hurt); the senior may have to own test generation.
- The sequential/parallel ratio per bin for *coding* tasks — Snell measured MATH only and did not combine search with revisions.
- The browser gate's cost and flakiness make it unsuitable inside the inner loop; whether to run it only on the final cluster is undecided.
- Fitted a, b values from Monkeys and exact per-budget accuracies from Snell were not in the fetched text; local fits must start from scratch.
- Temperature for plan sampling vs. implementation sampling on local quantised models (papers span 0.25–0.8).

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2310.04406	Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models	2023	ICML 2024	https://github.com/lapisrocks/LanguageAgentTreeSearch	https://arxiv.org/pdf/2310.04406
2305.10601	Tree of Thoughts: Deliberate Problem Solving with Large Language Models	2023	NeurIPS 2023	https://github.com/princeton-nlp/tree-of-thought-llm	https://arxiv.org/pdf/2305.10601
2203.11171	Self-Consistency Improves Chain of Thought Reasoning in Language Models	2022	ICLR 2023	—	https://arxiv.org/pdf/2203.11171
2207.10397	CodeT: Code Generation with Generated Tests	2022	ICLR 2023	https://github.com/microsoft/CodeT	https://arxiv.org/pdf/2207.10397
2203.07814	Competition-Level Code Generation with AlphaCode	2022	Science 378(6624)	https://github.com/deepmind/code_contests (data)	https://arxiv.org/pdf/2203.07814
2407.21787	Large Language Monkeys: Scaling Inference Compute with Repeated Sampling	2024	arXiv / CoRR	https://github.com/ScalingIntelligence/large_language_monkeys	https://arxiv.org/pdf/2407.21787

arXiv id	Title	Year	Venue	Code	PDF
2408.03314	Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters	2024	ICLR 2025 (oral)	—	https://arxiv.org/pdf/2408.03314